

LADiM: LAYERED DIAGNOSIS FOR MULTI-AGENT CODE MIGRATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Migrating deep learning code across frameworks is a practical requirement for reusing training programs in new software and hardware environments. Framework differences can change how a translated program computes gradients and updates its parameters without causing runtime errors. Such semantic drift may remain hidden even when the program runs successfully and its loss initially agrees with the source. We propose LaDiM, a multi-agent framework that uses dependencies between training computations to guide diagnosis and repair. The Verifier investigates where the translated program diverges from the source and passes supporting evidence to a separate Repair Agent. Measurements after each submitted change guide further repair. For repositories, LaDiM tracks dependencies between files and retains evidence as work moves between them. On 50 migration tasks from PyTorch to MindSpore, LaDiM matches MatchFixAgent’s full acceptance with 57.4% fewer total tokens, including initial translation. Experiments on MindSpore and JAX, together with migration tasks from Java to Python, demonstrate LaDiM’s effectiveness across frameworks and programming languages.

1 INTRODUCTION

Migrating deep learning code allows existing models and training programs to run in new software and hardware environments. Differences in operator semantics, numerical precision, and random number handling can change the computations performed after migration. Automatic differentiation and optimizer interfaces introduce further differences in how a model learns (Paszke et al., 2019; MindSpore, 2020). Large language models (LLMs) reduce the manual effort involved in code translation (He et al., 2025; Macedo et al., 2025b), while automated tests and interactive repair help assess and correct translated behavior (Rozière et al., 2022; Yang et al., 2024; Ibrahimzada et al., 2025). Training programs require these capabilities to preserve the computations that determine each parameter update.

A translated training program can produce a loss close to that of the source while computing incorrect gradients. An error in the optimizer can also alter the parameter update without changing the current loss or gradients. We use semantic drift to describe such changes in training behavior caused by migration. Figure 1 illustrates these cases by perturbing gradients or suppressing part of an update. The corresponding measurements detect each fault at the first step, whereas the mean loss difference crosses the detection threshold later. These dependencies also matter for diagnosis because an error in one computation can affect several downstream measurements. A forward error may change the gradient and update, and an incorrect update may affect later forward computations. Locating the cause therefore requires examining how the translated code produces the observed differences.

We introduce LaDiM (Layered Diagnosis for Multi-Agent Code Migration), which uses dependencies between training computations to guide investigation of an initial translation. The Verifier inspects code and runs tests to establish where source and target behavior diverge. For example, matching forward values with missing gradients directs attention to parameter registration and the differentiation path. The Verifier records the code and measurements supporting its hypotheses and passes this evidence to a separate Repair Agent. After the Repair Agent submits a change, the Orchestrator runs the acceptance checks and returns new measurements to guide subsequent attempts. For repositories, this process must also account for shared implementations and multi-

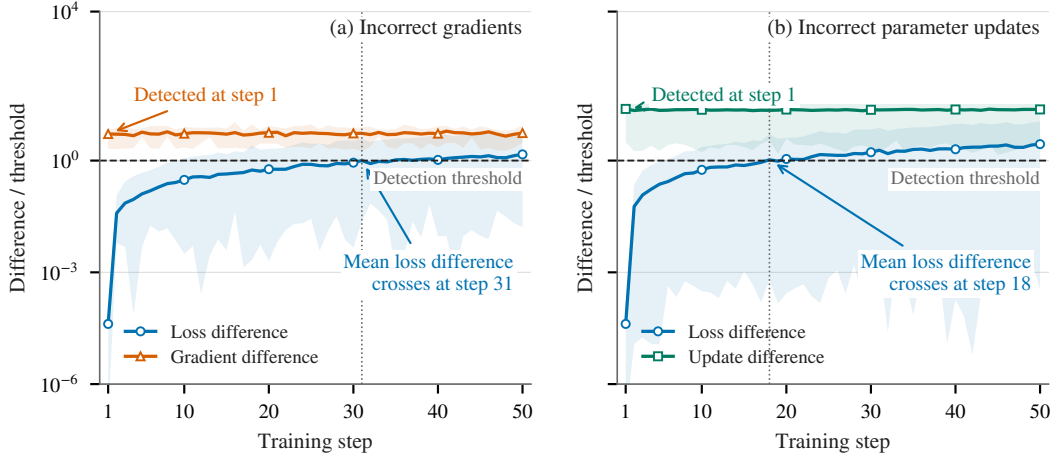


Figure 1: Loss and gradient differences (a) and loss and update differences (b), normalized by each signal’s threshold. The faults scale gradients in (a) and suppress part of the parameter update in (b). Gradient and update checks detect their respective faults at step 1 in all 12 runs per panel. The mean loss difference crosses the detection threshold (dashed) at steps 31 and 18, respectively. Curves show means and shading shows ranges.

ple entry points. LaDiM tracks dependencies between work units, invalidates checks affected by changes, and preserves retrievable evidence as the investigation moves across files.

We evaluate migration acceptance and model use from common initial translations under a shared evaluation protocol. On 50 tasks from PyTorch to MindSpore, LaDiM matches MatchFixAgent’s full acceptance with 57.4% fewer total tokens. The savings cover both repair of faulty translations and confirmation of candidates that already pass the checks. On a time series repository, all three evaluated agents complete the migration, with LaDiM using fewer tokens and repair calls. A recommendation repository exposes remaining update and representation differences even after LaDiM passes the loss, gradient, and original test checks. Training signal ablations examine which observations bring faults into the repair process, and experiments with JAX and Java-to-Python migration assess effectiveness across frameworks and languages. Our contributions are as follows.

- We formulate training-code migration around execution, forward values, gradients, and parameter updates, using their dependencies to investigate unknown semantic faults.
- We develop LaDiM with independent diagnosis, explicit evidence handoff, and verification after repair, together with dependency and evidence management for repositories.
- We evaluate migration across frameworks and languages, comparing complete acceptance and token use and analyzing repair behavior and training signal coverage.

2 RELATED WORK

Code Translation and Framework Migration. Neural translation methods learn code mappings from multilingual corpora and use automated tests or compiler representations to improve translations (Lachaux et al., 2020; Rozière et al., 2022; Szafraniec et al., 2023). ExeCoder encodes functional semantics, syntax structure, and variable dependencies (He et al., 2025), while InterTrans explores intermediate translation paths (Macedo et al., 2025b). Framework migration tools such as MindConverter supply mappings between programming interfaces (MindSpore, 2023). RepoTransAgent uses repository context, test failures, and reflection to refine translated code (Guan et al., 2025). LaDiM investigates how a translated training program computes and updates its state. At repository scale, it coordinates changes to shared interfaces and checks their effect across declared entry points.

Differential Testing and Deep Learning Verification. Differential testing exposes compiler and program faults through disagreement between related executions (Yang et al., 2011; Le et al., 2014). DeepXplore and DeepGauge investigate behavioral differences and test coverage in deep learning systems (Pei et al., 2017; Ma et al., 2018). More directly related to framework migration, TensorScope tests corresponding APIs across frameworks to identify inconsistent behavior (Deng et al., 2023). NablaFuzz compares execution scenarios to test automatic differentiation, including first-order and higher-order gradients (Yang et al., 2023). LaDiM uses training differences as evidence for code repair. Its Verifier relates the measured forward values, gradients, and parameter updates to the implementation, develops hypotheses about their origin, and supplies evidence that the Repair Agent can test when selecting a change.

Automated Program Repair and LLM Agents. Automated program repair uses tests or semantic constraints to guide patch generation (Nguyen et al., 2013), and pretrained language models generate patches from code and failure evidence (Xia et al., 2023). Interactive agents combine repository navigation, editing, and execution (Yang et al., 2024; Shinn et al., 2023; Chen et al., 2024b; Wang et al., 2024). Multi-agent systems distribute development activities among specialized roles (Qian et al., 2024; Hong et al., 2024; Wu et al., 2024; Chen et al., 2024a). MatchFixAgent takes a source program and an existing translation and uses several semantic analyses to guide test generation and repair (Ibrahimzada et al., 2025). LaDiM includes initial translation and organizes diagnosis around dependencies in training computations. The independent investigation and its supporting observations pass to a separate Repair Agent, which uses measurements after each submission to revise its hypotheses. The comparison evaluates both methods from the same initial translation and includes translation costs.

3 LADiM

LaDiM migrates a source training program into the requested language and framework through diagnosis and repair of an initial translation. The Translator generates a candidate, and the Verifier inspects its code and compares its training behavior with the source. A separate Repair Agent examines the resulting evidence to choose and test changes. These three roles use an LLM. The Orchestrator schedules their work and evaluates each submitted candidate, returning updated measurements for investigating remaining discrepancies. Figure 2 shows this process, which continues until the candidate passes the acceptance checks or reaches the task budget.

3.1 TASK DEFINITION AND TRAINING OBSERVATIONS

Given a source training program P in language L_s and framework F_s , migration produces a program $\hat{P}$ for the target language and framework (L_t, F_t) that preserves its training behavior. The task specification identifies the target environment and the files that may be modified. Depending on the task, edits may affect the translated program or a library implementation it uses. A supplied translation can enter directly at the investigation stage. The evaluation protocol establishes corresponding inputs and initial parameters, defines the tensor and parameter mappings, and specifies the observations and tolerances used for acceptance.

Execution adapters collect source and target measurements in a common format. At training step t , we write

$$\mathcal{O}_t(P) = (e_t, v_t, g_t, u_t), \quad (1)$$

where e_t records execution status, v_t contains selected forward values and losses, g_t contains applicable gradients, and u_t contains parameter updates. These observations follow the dependencies within a training step. The forward computation produces the values used to evaluate the loss. Differentiating this loss yields the gradients used by the optimizer to update the parameters. Some tasks also expose buffers or optimizer state. If execution fails, the adapter returns the exception and observations already obtained. Acceptance uses the complete comparisons required for the task, including checks on shapes, data types, and trainability. Section 4.1 specifies which observations apply to each collection and how state is maintained across training steps.

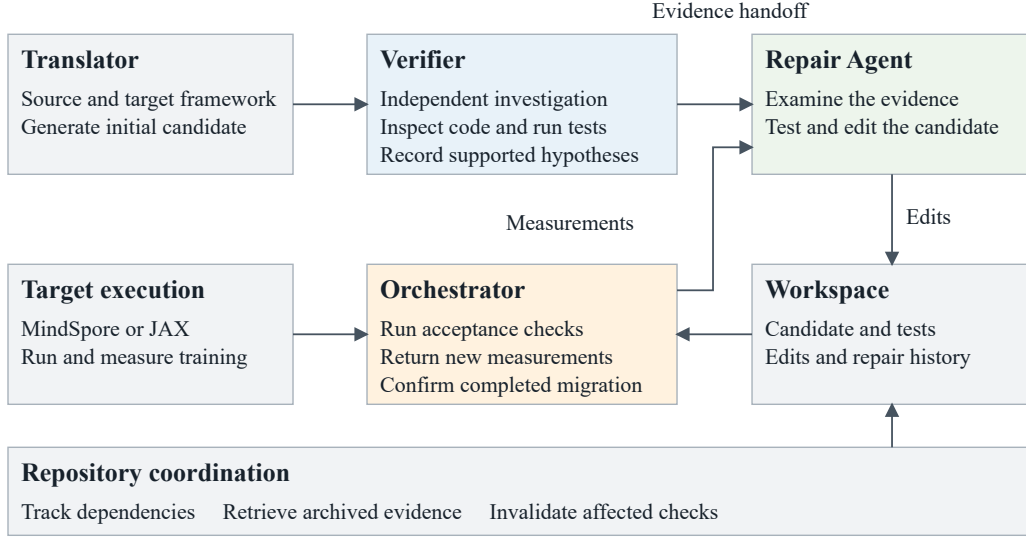


Figure 2: LaDiM’s migration workflow. The Translator generates a candidate for the Verifier to investigate through code and training observations. The Repair Agent receives the evidence and revises the candidate. The Orchestrator checks submitted changes and returns measurements for subsequent attempts. Execution adapters obtain observations from the target framework.

3.2 AUTONOMOUS LAYERED DIAGNOSIS

The Verifier uses the dependencies between training computations to narrow its investigation. An execution failure supplies an exception and any reported code location. When forward values differ, the Verifier examines whether the same error explains later gradient and update differences. If forward values agree, a gradient discrepancy directs attention to differentiation. Agreement in gradients with different parameter updates motivates investigation of the optimizer. The training step at which a discrepancy first appears also matters because updates affect later forward computations. These relationships guide hypotheses that the Verifier checks against the code and additional tests.

During diagnosis, the Verifier can read the source, candidate, and relevant library code and create temporary tests. Production files remain read-only. Consider a parameter whose source gradient is present but whose target gradient is missing. The Verifier can inspect parameter registration, trace the operations connecting that parameter to the loss, and run a small test to distinguish an unregistered parameter from a disconnected computation. It records the measured outcome and code location supporting a hypothesis, along with unresolved questions. This record can include a further test to distinguish competing explanations. Fault categories and repair locations emerge from this investigation. They are not supplied as task inputs. The recorded code observations and test results give the Repair Agent enough context to reassess the diagnosis before editing.

3.3 EVIDENCE HANDOFF AND VERIFIED REPAIR

The Repair Agent begins a separate conversation with the Verifier’s recorded investigation, including its code observations, test results, and final diagnosis. The handoff attributes these records to the Verifier, so the Repair Agent can examine the evidence behind a hypothesis and choose further tests before changing the code. It retains the same inspection and testing tools and gains permission to edit the files specified by the task. Each edit undergoes syntax validation before it is applied to the workspace. This separation makes the initial diagnosis an explicit input to repair and allows the effect of independent evidence handoff to be compared with investigation and editing in one conversation.

After each submission, the Orchestrator evaluates the current candidate and returns the measured discrepancies. The Repair Agent compares them with earlier observations to assess the change

and decide which hypothesis or code location to investigate next. The current files and repair conversation persist across attempts, and all calls count against one cumulative task budget. Progress reminders report the remaining allowance. When all required checks and confirmation runs pass, the Orchestrator accepts the candidate. A candidate that initially passes the checks can finish investigation and confirmation without corrective edits. If the budget is exhausted, LaDiM returns the current candidate and its verification status.

3.4 REPOSITORY COORDINATION

Repository migration requires changes to shared implementations to remain compatible with several callers. LaDiM provides a map of source files, imports, symbols, and notebook cells from which the agent proposes work units with explicit file scopes and dependencies. Each unit describes an interface goal and any declared local tests. The agent selects a unit whose prerequisites have current checkpoints and investigates the related interfaces before editing. A checkpoint records syntax validation and the outcome of any selected tests. When code changes, previous measurements are marked as belonging to an earlier candidate, and checkpoints for affected units and their dependents become stale. Revising a plan can also invalidate local checkpoints, while measurements retain their association with the unchanged candidate. The Orchestrator determines final acceptance by evaluating the complete declared repository scope, including its entry points and training checks.

As work moves across files, LaDiM keeps complete observations and conversations in an evidence archive outside the editable workspace. A read-only tool retrieves records or selected portions of them, allowing an agent to inspect arrays or logs omitted from a summary. At the independent role handoff, an actual switch between work units, or a context capacity boundary, LaDiM rebuilds the conversation around the current plan and available evidence. It retains the full structured hypotheses and next steps, latest measurements, and recently read code for the active unit whose file hashes still match. First selecting a unit and continuing an ordinary repair stage preserve the existing conversation. These rules carry the state of the investigation between units while keeping the evidence associated with the code it describes. Appendix F gives the context limits and stage budgets.

4 EXPERIMENTS

4.1 EXPERIMENTAL SETUP

Tasks and verification. The main MindSpore collection contains 50 migration tasks covering tensor operations, CNNs, image MLPs, Transformer components, small language models, and parameter and optimizer interfaces. The comparison across languages contains 18 Java/DJL programs translated to Python/PyTorch, comprising eight textbook examples and ten applications. We also migrate a time series repository and a recommendation repository from PyTorch to MindSpore. Every method in a collection receives the same source programs and task requirements, with common evaluation inputs and corresponding initial parameters. Acceptance requires all applicable training comparisons and execution checks to pass. The individual programs retain their own state over two consecutive steps, and the repositories use three steps. Seed 101 supplies public feedback, with seeds 202 and 303 used for confirmation after the public checks pass. Operator tasks compare forward values and applicable input gradients. Model tasks also check parameter gradients and, when a training entry point is present, parameter updates. Observable optimizer state is checked where specified. Repository acceptance additionally requires the declared scripts, notebooks or original tests, together with file coverage and documentation checks. Appendix A details task construction, numerical tolerances, and the observations covered by each protocol.

Methods and budgets. We compare LaDiM with SWE-agent (Yang et al., 2024) and MatchFix-Agent (Ibrahimzada et al., 2025) from the same initial translations, also evaluated as Direct LLM. The MindSpore comparison includes CodeTransEngine’s direct translation configuration (Macedo et al., 2025a) and MSAdapter (OpenI, 2025). The comparison across languages includes InterTrans (Macedo et al., 2025b) and a repair loop that rewrites the candidate using test feedback. Native algorithms, prompts, tools, parsers, and retry policies are retained, with the same DeepSeek backend for model calls. Each individual migration allows up to 40 repair calls, 120,000 output tokens, 1,800 seconds, and four external submissions. Repository limits are 80 calls, 480,000 output tokens, 3,600

Table 1: Migration results and model use on common inputs. Panels (a) and (b) report accepted programs and complete token costs. Panels (c) and (d) report checks passed and repository costs. Time series checks cover three seeds. Recommendation checks use the public seed.

Method	Accepted	Model calls	Input tokens (millions)	Output tokens (millions)	Total tokens (millions)	
(a) PyTorch to MindSpore						
Direct LLM	29/50	29	0.018	0.411	0.429	
CodeTransEngine (direct)	31/50	29	0.022	0.257	0.280	
MSAdapter	15/50	0	0.000	0.000	0.000	
SWE-agent	44/50	1,097	19.845	0.987	20.831	
MatchFixAgent	50/50	619	10.750	1.347	12.097	
LaDiM (ours)	50/50	326	4.298	0.861	5.159	
(b) Java/DJL to Python/PyTorch						
Direct LLM	1/18	18	0.142	0.552	0.694	
InterTrans	1/18	89	0.680	1.606	2.286	
Test-guided repair	7/18	71	1.594	2.881	4.475	
SWE-agent	8/18	682	23.063	1.258	24.321	
MatchFixAgent	8/18	524	26.335	2.020	28.355	
LaDiM (ours)	9/18	345	19.470	2.009	21.479	
(c) Time series repository						
Method	Paired checks	Gradients	Parameter updates	Entry points	Repair calls	Total tokens (millions)
SWE-agent	69/69	9/9	9/9	3/3	80	8.200
MatchFixAgent	69/69	9/9	9/9	3/3	69	13.598
LaDiM (ours)	69/69	9/9	9/9	3/3	35	2.614
(d) Recommendation repository						
Method	Paired checks	Gradients	Parameter updates	Original tests	Repair calls	Total tokens (millions)
SWE-agent	104/145	17/18	12/18	10/10	80	8.317
MatchFixAgent	129/145	18/18	16/18	n/a	80	8.679
LaDiM (ours)	130/145	18/18	15/18	10/10	80	8.811

seconds, and four submissions. These limits include diagnosis. A submission follows the method’s internal actions, which may contain several model calls and edits. Appendix A.2 gives per-call limits and method integration settings.

Cost and reporting. Migration costs include one copy of the common initial translation and every subsequent model call, including failed attempts. We count input and output tokens, retaining cached input tokens reported by the provider. Functional failures and incomplete runs remain in the declared task denominator and cost totals. Acceptance at a budget scores the current candidate at that checkpoint. For repositories, we report complete acceptance separately from the number of declared paired checks passed. Those checks include numerical comparisons, execution, and source behavior. Measurements unavailable after execution failure are n/a and remain in the declared check denominator. The natural fault analysis starts from supplied candidates and reports repair costs for the selected complete runs.

4.2 MAIN EXPERIMENTS

4.2.1 MIGRATION ACROSS FRAMEWORKS

LaDiM completes the MindSpore collection with less than half the tokens used by MatchFixAgent at the same acceptance level (Table 1a). Direct LLM, CodeTransEngine, and MSAdapter use fewer tokens but leave part of the collection unresolved. SWE-agent requires more tokens than LaDiM and achieves lower acceptance. LaDiM’s savings cover both corrective work and confirmation of

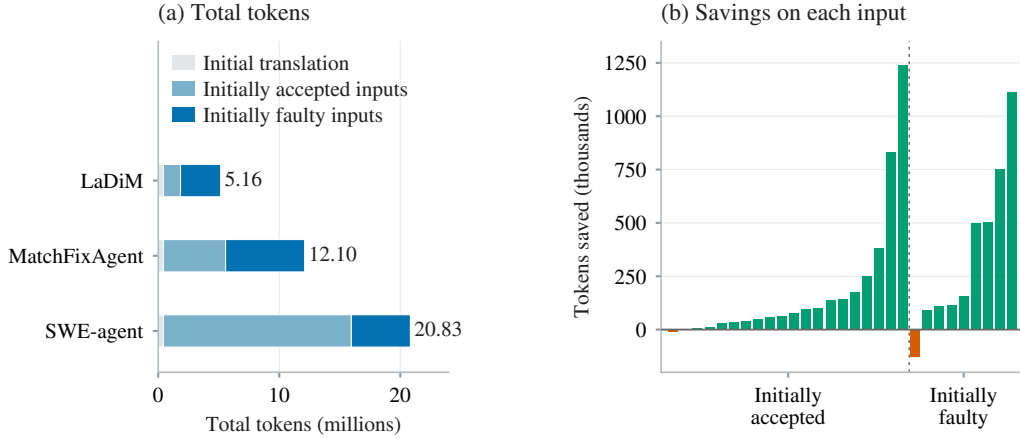


Figure 3: Token use on the common MindSpore collection. (a) Totals include the shared initial translation and all subsequent calls, with subsequent costs grouped by whether the initial candidate passes the acceptance checks. (b) Each bar shows MatchFixAgent tokens minus LaDiM tokens for one migration input. Bars are sorted within each group, and positive values indicate lower token use by LaDiM. LaDiM uses fewer tokens on 19 of 20 initially accepted inputs and 8 of 9 initially faulty inputs.

candidates that already pass the checks. After the common initial translation, diagnosis and repair on faulty inputs use 49.2% fewer tokens than MatchFixAgent, while investigation and confirmation on initially accepted inputs use 72.6% fewer. LaDiM also uses fewer tokens on 27 of the 29 distinct migration inputs, so the total savings extend across most inputs (Figure 3).

LaDiM reaches 46/50 acceptance after one submission and 50/50 after two, retaining full acceptance at the four-submission budget. Seven of the nine distinct faulty inputs are repaired after the first submission. New measurements guide the remaining two corrections after that submission. MatchFixAgent reaches full acceptance within its first external submission through its internal analysis and repair orchestration. Its complete migration process uses 619 model calls, compared with LaDiM’s 326. Input tokens account for 93.0% of the total token reduction against MatchFixAgent.

4.2.2 MIGRATION ACROSS LANGUAGES

LaDiM completes all eight textbook examples and one application program in the Java/DJL collection (Table 1b). Its accepted set contains every program accepted by MatchFixAgent and one additional recurrent network example. The common initial translation accepts only the application program, which all repair methods preserve. Repair therefore adds coverage across the training examples, where corresponding interfaces permit direct comparison of the training computations. LaDiM achieves the highest acceptance among the compared methods while using 11.7% fewer tokens than SWE-agent and 24.2% fewer than MatchFixAgent.

4.2.3 REPOSITORY MIGRATION

All three agents pass 69/69 paired checks and execute all three entry points on the time series repository, where a shared model is used by the training script, notebook, and teaching examples (Table 1c). LaDiM completes repair in 35 model calls, compared with 80 for SWE-agent and 69 for MatchFixAgent. Its total token use is 68.1% and 80.8% lower, respectively. This result extends the efficiency comparison to a repository where changes to shared code must work across several entry points. The measured costs cover the whole repository workflow described in Section 3.4.

On the recommendation repository, LaDiM passes 130/145 paired checks, compared with 104/145 for SWE-agent and 129/145 for MatchFixAgent (Table 1d). LaDiM passes all ten original tests, executes the training command, and satisfies file coverage and documentation checks. It passes

99/114 numerical checks, compared with 74/114 for SWE-agent and 99/114 for MatchFixAgent. The two equal numerical counts cover different checks. LaDiM completes reward model inference and retrieval, while MatchFixAgent passes one additional parameter update check. MatchFixAgent also leaves original tests unable to run because of a remaining source framework import, and fails file coverage and documentation checks. Appendix E reports the full comparison, including unavailable measurements.

The remaining LaDiM discrepancies arise despite agreement in all 18 forward loss and all 18 gradient comparisons. Parameter updates pass 15/18 checks, while item representations pass 6/18. The representation differences occur at the second and third training steps across all six model paths and account for 12 of the 15 numerical discrepancies. As in Figure 1, agreement in loss and gradients does not establish agreement in the evolving training state. Direct comparison of updates and subsequent representations exposes the remaining behavior to investigate.

4.3 ANALYSIS EXPERIMENTS

4.3.1 REPAIRING NATURAL TRANSLATION FAULTS

We evaluate ten supplied translations comprising five initially faulty programs and five initially accepted programs, with edits allowed in the candidate and its supporting library. LaDiM repairs four faulty programs and preserves all five accepted programs (Table 2). The three repair baselines preserve acceptance but leave all five faulty programs unresolved. Thus the difference comes from producing effective corrections on faulty translations. This collection requires investigation of both translated code and the library operations used during training. Its repair costs include unsuccessful attempts on the remaining faulty program as well as investigation of initially accepted programs. The direct repair control uses the fewest tokens, while LaDiM’s additional investigation and repair produce four more accepted programs.

One successful repair begins with an LSTM translation that cannot execute because the supporting library has no registered implementation for its recurrent operator. LaDiM inspects the dispatch registration and reproduces the failure, then adds the recurrent computation using operations in the target framework. The next evaluation proceeds beyond the encoder but exposes recursive tensor indexing. A focused indexing test identifies the dispatch cycle, and a further edit restores execution and agreement in the measured loss, gradients, and parameter updates. The sequence illustrates how measurements after a change reveal the next fault to investigate. Appendix D records the checks for this case.

Table 2: Repair of natural translation faults and preservation of initially accepted programs. Tokens cover all ten selected complete repair runs, starting from supplied translations.

Method	Faulty programs repaired	Accepted programs retained	Final accepted	Repair tokens (millions)
Direct repair (shared tools)	0/5	5/5	5/10	3.751
SWE-agent	0/5	5/5	5/10	12.678
MatchFixAgent	0/5	5/5	5/10	14.351
LaDiM	4/5	5/5	9/10	13.207

4.3.2 TRAINING SIGNALS AND DETECTION LATENCY

The diagnostic study follows four models for 50 steps under three seeds. With per-step synchronization, each target model starts the checked step from the source state, allowing a discrepancy to be associated with the current computation. Correct translations pass the recorded checks, with a maximum loss difference of 3.815×10^{-6} . Gradient faults can preserve loss agreement, and update faults can preserve both the current forward computation and gradient. When each execution retains its own state, the faults influence later forward values. The gradient and update checks in Figure 1 detect their respective faults at step 1 in all 12 runs per panel. Individual loss trajectories cross the threshold within the observation window in only 3/12 and 4/12 runs, even though the mean curves cross at steps 31 and 18. Loss detection therefore depends on how errors accumulate, while direct gradient and update checks make the affected computation available for immediate investigation.

4.3.3 GENERALIZATION ACROSS FRAMEWORKS

The JAX study applies the same diagnosis and repair procedure to six MLP and CNN faults. LaDiM and the direct repair control with shared tools both accept all six within their first submission under verification on three seeds. The execution adapter changes while the investigation continues to use corresponding forward values, gradients, and parameter updates. These repairs demonstrate LaDiM’s effectiveness with another target framework. Appendix A reports the implementation and complete costs.

4.4 ABLATION STUDIES

Training signal coverage. We vary the default verification feedback on a CNN, image MLP, Transformer classifier, and small causal language model. Each has an execution, forward value, gradient, and parameter update fault, giving 16 instances per setting. The conditions share sources, candidates, tools, and budgets and allow temporary tests. Available feedback controls when repair starts and stops. Every final candidate is then scored with complete checks on three seeds.

Table 3: Effect of training signal coverage on detection and repair. Accepted candidates pass final full verification. Faults passing the initial available checks trigger no repair.

Available feedback	Accepted	Faults passing initial checks	Total tokens (millions)
Execution and basic checks	4/16	12	2.089
+ Forward values	8/16	8	2.603
+ Gradients	12/16	4	3.303
+ Parameter updates	16/16	0	3.419

Adding each signal brings another class of faults into repair, increasing final acceptance from 4/16 with execution and basic checks to 16/16 with complete feedback (Table 3). Forward comparisons expose changed values. Gradient checks reveal faults in differentiation, and update checks expose incorrect optimizer behavior. Under partial feedback, the omitted faults satisfy the available checks and trigger no repair. Complete feedback exposes all 16 faulty programs, which subsequently pass final verification. The results show how training observations affect the whole detection and repair process.

Evidence handoff and repair context. Independent evidence handoff achieves 50/50 on the main collection, compared with 49/50 when investigation and repair continue in one conversation. Removing prior repair conversation or progress reminders retains full acceptance and reduces tokens. Most inputs finish after confirmation or one corrective submission, so few attempts use earlier repair history. On natural translation faults, the reference configuration with editing format assistance repairs 4/5 faulty programs when repair history is retained and 0/5 when it is removed. Both settings preserve all five initially accepted programs. Appendix B reports the complete component comparison and integration into native agent hosts.

5 CONCLUSION

We introduced LaDiM, a multi-agent framework for migrating deep learning code while preserving training behavior. Specifically, LaDiM uses dependencies between training computations to guide diagnosis and passes the resulting evidence to a separate Repair Agent. Verification after each submission guides subsequent changes, and dependency tracking with evidence retrieval supports repair across files. This design helps detect and repair semantic errors that remain hidden when evaluation checks only execution or loss. Experiments across frameworks and programming languages demonstrate LaDiM’s effectiveness in preserving training behavior and its efficiency in diagnosis and repair.

AI USE STATEMENT

REFERENCES

- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, Yujia Qin, Xin Cong, Ruobing Xie, Zhiyuan Liu, Maosong Sun, and Jie Zhou. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *Proceedings of the Twelfth International Conference on Learning Representations*, 2024a. URL <https://openreview.net/forum?id=EHg5GDnyq1>.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *Proceedings of the Twelfth International Conference on Learning Representations*, 2024b. URL <https://openreview.net/forum?id=KuPixIqPiQ>.
- Zizhuang Deng, Guozhu Meng, Kai Chen, Tong Liu, Lu Xiang, and Chunyang Chen. Differential testing of cross deep learning framework APIs: Revealing inconsistencies and vulnerabilities. In *32nd USENIX Security Symposium (USENIX Security 23)*, pp. 7393–7410. USENIX Association, 2023. URL <https://www.usenix.org/conference/usenixsecurity23/presentation/deng-zizhuang>.
- Google. torchax: A PyTorch frontend for JAX. <https://google.github.io/torchax/>, 2025. Accessed: 2026-06-02.
- Ziqi Guan, Xin Yin, Zhiyuan Peng, and Chao Ni. RepoTransAgent: Multi-agent LLM framework for repository-aware code translation. *arXiv preprint arXiv:2508.17720*, 2025. doi: 10.48550/arXiv.2508.17720. URL <https://arxiv.org/abs/2508.17720>.
- Minghua He, Yue Chen, Fangkai Yang, Pu Zhao, Wenjie Yin, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. ExeCoder: Empowering large language models with executability representation for code translation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 7099–7125, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.362. URL <https://aclanthology.org/2025.emnlp-main.362/>.
- Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *Proceedings of the Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VtmBAGCN7o>.
- Ali Reza Ibrahimzada, Brandon Paulsen, Reyhaneh Jabbarvand, Joey Dodds, and Daniel Kroening. MatchFixAgent: Language-agnostic autonomous repository-level code translation validation and repair, 2025. arXiv:2509.16187.
- Marie-Anne Lachaux, Baptiste Rozière, Lowik Chanussot, and Guillaume Lample. Unsupervised translation of programming languages. In *Advances in Neural Information Processing Systems*, volume 33, pp. 20601–20611, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/ed23fbf18c2cd35f8c7f8de44f85c08d-Abstract.html>.
- Vu Le, Mehrdad Afshari, and Zhendong Su. Compiler validation via equivalence modulo inputs. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp. 216–226. ACM, 2014. doi: 10.1145/2594291.2594334. URL <https://doi.org/10.1145/2594291.2594334>.
- Lei Ma, Felix Juefei-Xu, Fuyuan Zhang, Jiyuan Sun, Minhui Xue, Bo Li, Chunyang Chen, Ting Su, Li Li, Yang Liu, Jianjun Zhao, and Yadong Wang. DeepGauge: Multi-granularity testing criteria for deep learning systems. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, pp. 120–131. ACM, 2018. doi: 10.1145/3238147.3238202. URL <https://doi.org/10.1145/3238147.3238202>.
- Marcos Macedo, Yuan Tian, and Bram Adams. CodeTransEngine: Ready-to-use backend for LLM-based code translation. In *ICLR 2025 Third Workshop on Deep Learning for Code*, 2025a. URL <https://openreview.net/forum?id=9WHJuzjPJf>.

- Marcos Macedo, Yuan Tian, Pengyu Nie, Filipe R. Cogo, and Bram Adams. InterTrans: Leveraging transitive intermediate translations to enhance LLM-based code translation. In *Proceedings of the 47th International Conference on Software Engineering*, pp. 2819–2831. IEEE, 2025b. doi: 10.1109/ICSE55347.2025.00236. URL <https://doi.org/10.1109/ICSE55347.2025.00236>.
- MindSpore. MindSpore: A new deep learning computing framework. <https://www.mindspore.cn/en>, 2020. Framework white paper and documentation.
- MindSpore. Migrating from third party frameworks with MindConverter. https://www.mindspore.cn/mindinsight/docs/en/r2.0.0-alpha/migrate_3rd_scripts_mindconverter.html, 2023. Accessed: 2026-06-02.
- Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. SemFix: Program repair via semantic analysis. In *Proceedings of the 35th International Conference on Software Engineering*, pp. 772–781. IEEE, 2013. doi: 10.1109/ICSE.2013.6606623. URL <https://doi.org/10.1109/ICSE.2013.6606623>.
- OpenI. MSAdapter. <https://openi.pcl.ac.cn/OpenI/MSAdapter/src/tag/v0.6.0>, 2025. Version 0.6.0. Accessed: 2026-09-20.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pp. 8024–8035, 2019. URL <https://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>.
- Kexin Pei, Yinzhi Cao, Junfeng Yang, and Suman Jana. DeepXplore: Automated whitebox testing of deep learning systems. In *Proceedings of the 26th Symposium on Operating Systems Principles*, pp. 1–18. ACM, 2017. doi: 10.1145/3132747.3132785. URL <https://doi.org/10.1145/3132747.3132785>.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pp. 15174–15186. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.810. URL <https://aclanthology.org/2024.acl-long.810/>.
- Baptiste Rozière, Jie M. Zhang, Francois Charton, Mark Harman, Gabriel Synnaeve, and Guillaume Lample. Leveraging automated unit tests for unsupervised code translation. In *Proceedings of the Tenth International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=cmt-6KtR4c4>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2303.11366>.
- Marc Szafraniec, Baptiste Rozière, Hugh James Leather, Patrick Labatut, Francois Charton, and Gabriel Synnaeve. Code translation with compiler representations. In *Proceedings of the Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=XomEU3eNeSQ>.
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 50733–50749, Vienna, Austria, 2024. PMLR. URL <https://proceedings.mlr.press/v235/wang24h.html>.

- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *Proceedings of the First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=uAjxFFing2>.
- Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering*, pp. 1482–1494. IEEE, 2023. doi: 10.1109/ICSE48619.2023.00129. URL <https://doi.org/10.1109/ICSE48619.2023.00129>.
- Chenyuan Yang, Yinlin Deng, Jiayi Yao, Yuxing Tu, Hanchi Li, and Lingming Zhang. Fuzzing automatic differentiation in deep-learning libraries. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 2023. doi: 10.1109/ICSE48619.2023.00105. URL <https://arxiv.org/abs/2302.04351>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024. doi: 10.48550/arXiv.2405.15793. URL <https://arxiv.org/abs/2405.15793>.
- Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation*, pp. 283–294. ACM, 2011. doi: 10.1145/1993498.1993532. URL <https://doi.org/10.1145/1993498.1993532>.

A ADDITIONAL EXPERIMENTAL PROTOCOLS

A.1 TASK CONSTRUCTION AND ACCOUNTING

The main MindSpore collection contains 50 task identifiers. Twelve entries reuse existing native source programs, and the remaining entries use programs authored from public interfaces and declared semantics. The identifiers correspond to 29 distinct combinations of source program and evaluation settings drawn from 24 source files. Conditions with identical source, evaluation settings, initial candidate, method, and inputs are executed once and associated with the corresponding identifiers. Acceptance in the main table uses all 50 identifiers, while token totals count actual calls. LaDiM, SWE-agent, and MatchFixAgent share the same initial translation for each distinct input. Production edits are restricted to the candidate program, with separate temporary tests available for investigation. Sources and task specifications remain fixed. Each accepted candidate passes all applicable checks at seeds 101, 202, and 303.

The comparison across languages contains ten DJL application programs and eight textbook examples. LaDiM accepts one application and all eight examples. SWE-agent and MatchFixAgent each accept the same application and seven examples, while test-guided repair accepts seven examples. The saved initial translations and their generation costs are shared by the repair conditions. InterTrans uses its direct Java-to-Python path and an intermediate JavaScript path, with native search and extraction procedures. Its 2,286,034 tokens include 574,304 from incomplete generation calls and 1,711,730 from the completed search. The final evaluation compares outputs, losses, gradients, parameter updates, and observable optimizer state over two consecutive training steps.

The common MindSpore initial translations cost 429,109 tokens. This amount is already included once in each repair method’s total. Additional LaDiM investigation uses 1,409,686 tokens on initially accepted inputs and 3,320,339 on initially faulty inputs. The corresponding MatchFixAgent totals are 5,135,863 and 6,532,453. Across the 29 distinct inputs, the median ratio of LaDiM to MatchFixAgent total tokens is 0.588. Shared initial translation costs cancel when their paired costs are subtracted.

A.2 NUMERICAL COMPARISONS AND MIGRATION SETTINGS

The main MindSpore collection uses absolute/relative tolerances of $10^{-4}/10^{-4}$ for forward values, $10^{-4}/10^{-3}$ for gradients, and $10^{-5}/10^{-3}$ for updates. BF16 matrix multiplication uses absolute tolerance 10^{-2} . Shapes, data types, initial values, and trainability must match. Parameter and optimizer tasks test their specified enumeration, gradient assignment, or update behavior. The source program and task requirements are available to each method. Acceptance also checks that the target framework executes the required computations.

SWE-agent retains its native tools and controller. MatchFixAgent retains its analysis and repair orchestration through a shared DeepSeek tool backend. CodeTransEngine generates its own translation through the native direct path, and MSAdapter uses its native interface mapping. LaDiM allows eight calls per investigation or repair stage on individual programs. Agent calls allow at most 16,384 output tokens, while initial translation and the repair loop that rewrites the complete candidate allow at most 131,072 per call. The latter has four repair rounds. All diagnosis and repair stages share the cumulative limits in Section 4.1. Repository calls allow at most 32,768 output tokens. Output budgets include reasoning tokens, and all input usage is recorded.

A.3 TRAINING SIGNAL AND FRAMEWORK STUDIES

The training signal and JAX studies use the same investigation and evidence handoff procedure with editing examples and format-correction feedback in their tool prompts. The signal experiment applies four controlled modifications to four model classes. The execution fault uses an invalid optimizer keyword, the forward fault adds a constant loss offset, the gradient fault scales gradients, and the update fault changes the learning rate. Each setting retains the initial investigation, current workspace, and cumulative call budget. Its visible checks determine whether repair starts and stops. Final acceptance uses all training signals on three seeds. MindSpore measurements use the `torch4ms` forward and backward bridge, followed by optimizer updates on its parameters.

The JAX study contains six faults in MLP and CNN candidate programs. TorchAX exposes the PyTorch interface over JAX (Google, 2025), with JAX computing gradients and Optax applying updates. Both LaDiM and direct repair allow four external submissions and accept every task at the first submission. LaDiM uses 674,009 tokens and direct repair uses 353,311. Native Ivy and torch2jax are also tested on these supplied faulty candidates and accept 0/6 repairs, while their healthy-input checks pass. These observations describe repair of supplied target candidates through JAX.

A.4 DETECTION MEASUREMENTS

The detection study uses four models, three seeds, and 50 training steps. Gradient scaling and partial update suppression are evaluated both with per-step synchronization to the source state and with each execution retaining its own state. Loss, gradient norm, and relative update thresholds are 0.02, 0.05, and 0.03. Figure 1 uses trajectories with separate state and reports the mean and range across 12 runs per fault. The loss threshold crossings at steps 31 and 18 refer to mean curves. Direct gradient or update checks detect all respective faults at step 1.

The scalar summaries are

$$\Delta_{\text{loss}} = |L(P) - L(\hat{P})|, \quad (2)$$

$$\Delta_{\text{grad}} = \left| \|\nabla_{\theta} L(P)\|_2 - \|\nabla_{\hat{\theta}} L(\hat{P})\|_2 \right|, \quad (3)$$

$$\Delta_{\text{param}} = \frac{\|\Delta\theta - \Delta\hat{\theta}\|_2}{\|\Delta\theta\|_2 + \epsilon}. \quad (4)$$

Here $\Delta\theta$ and $\Delta\hat{\theta}$ denote source and target parameter updates, and ϵ prevents division by zero. Figure 1 divides each difference by its detection threshold. The main migration comparison uses the task-specific tensor comparisons defined above.

B EVIDENCE HANDOFF AND AGENT INTEGRATION

The component study shares the main collection, initial translations, acceptance checks, and cumulative budgets. Continuing the original conversation retains its message roles and reasoning history when the agent begins editing. The independent handoff supplies the Verifier’s evidence to a separate repair conversation. Removing prior repair conversation restarts each subsequent repair attempt from the initial investigation and latest observation, while retaining the current files and cumulative budget. Removing progress reminders preserves the call and submission limits.

Table 4: Complete component and host-integration comparison on the common MindSpore collection. Token totals include initial translation and all subsequent calls.

Setting	Accepted	Total tokens (millions)
LaDiM	50/50	5.159
Continuous investigation and repair conversation	49/50	5.603
Without prior repair conversation	50/50	4.566
Without progress reminders	50/50	4.562
MatchFixAgent	50/50	12.097
MatchFixAgent with independent investigation	50/50	19.480
SWE-agent	44/50 [†]	20.831
SWE-agent with independent investigation	43/50	36.547

[†] Includes one task with unavailable final acceptance, which remains in the denominator and cost totals. Each condition uses the input reuse described in Appendix A.1.

LaDiM accepts all 50 identifiers, while the continuous conversation accepts 49. Removing prior repair conversation or progress reminders preserves full acceptance and uses fewer tokens. Twenty distinct initial translations are already accepted, and only two of the nine faulty pairs require a second repair submission under the complete method. Most cases therefore finish before a later repair attempt can use earlier repair conversation.

The natural translation component study uses the same ten supplied candidates as Table 2. Its reference configuration includes examples of the edit format and corrective feedback for malformed edits. Each condition changes the named component while retaining the candidates, workspace access, acceptance checks, and repair budgets. Removing this format assistance gives the LaDiM configuration reported in the natural fault comparison.

Table 5: Effect of repair context and editing assistance on natural translation faults. Tokens cover all ten selected complete runs.

Condition	Faulty programs repaired	Accepted programs retained	Final accepted	Repair tokens (millions)
Independent evidence handoff	4/5	5/5	9/10	15.548
Continuous conversation	3/5	5/5	8/10	14.516
Without repair history	0/5	5/5	5/10	10.852
Without progress reminders	2/5	5/5	7/10	17.325
Without editing format assistance	4/5	5/5	9/10	13.207

The host-integration conditions prepend the same read-only investigation to SWE-agent or MatchFixAgent. Investigation calls share the host’s total call, token, and time limits. The native host retains its own history and repair policy. MatchFixAgent with investigation retains 50/50 at a higher token total. SWE-agent with investigation accepts 43/50, compared with 44/50 and one interrupted task for its native condition. Across distinct inputs, the added investigation increases SWE-agent’s repair count from seven to eight among nine initially faulty programs. It also leaves four initially accepted programs failing at the end. Table 4 records both the repair and preservation consequences in final acceptance.

C ADDITIONAL REPAIR COLLECTIONS

We also evaluate supplied faulty target programs under their respective acceptance criteria. A controlled MindSpore collection contains 50 injected faults with task-specific acceptance checks. LaDiM accepts 45/50 within four submissions, compared with 29/50 for SWE-agent and 35/50 for a direct repair agent using shared tools. LaDiM’s cumulative accepted counts are 29, 41, and 45 at one, two, and four submissions. Total token use in the selected complete runs is 39,982,844, 42,579,064, and 12,459,013 for LaDiM, SWE-agent, and direct repair, respectively. The source and target inputs required by MatchFixAgent are supplied in the paired collection below.

In a separate collection of twelve source and target repair tasks, LaDiM accepts 12/12 and MatchFixAgent accepts 11/12. They use 6,642,603 and 14,505,833 tokens, respectively. Both receive the complete source, candidate, task specification, and target library. Each repair collection uses its specified editable code scope and acceptance checks. The corresponding agent and tool backend configurations are documented with the frozen sources and results.

D NATURAL TRANSLATION REPAIR PROTOCOL

The ten supplied translations contain five initially faulty programs and five that already pass the checks. Each method receives the same source, candidate, task requirements, and supporting library. The candidate and library may be edited. The source and acceptance criteria remain fixed. Target forward computation and differentiation execute in MindSpore through `torch4ms`, followed by PyTorch optimizer updates on its parameters. The comparison uses matching inputs and initial parameters with three-seed confirmation. Loss absolute difference, gradient norm absolute difference, and relative parameter update thresholds are 0.02, 0.05, and 0.03, with an additional gradient vector L2 threshold of 0.05 and the specified layer and parameter checks.

Table 2 counts every call in the selected complete run for each input, including failed repairs and investigation of initially accepted programs. LaDiM uses 193 calls, direct repair uses 120, SWE-agent uses 386, and MatchFixAgent uses 265. LaDiM’s selected runs use 13,206,863 tokens with complete usage records.

The LSTM example in Section 4.3 initially stops at an unregistered recurrent operator. Its first repair submission leaves execution unresolved. During the next attempt, a temporary test reproduces the failure and inspection identifies the missing dispatch entry. LaDiM adds a MindSpore recurrence and registers it in the supporting library. The second submission then exposes recursion in tensor indexing, which another test isolates. The third submission corrects the indexing dispatch and passes evaluation and confirmation. On the final public check, loss absolute difference is 2.38×10^{-7} , gradient vector L2 difference is 4.19×10^{-7} , and relative parameter update difference is 8.11×10^{-7} . The example uses 32 model calls. This trajectory belongs to the same LaDiM configuration as the natural fault table.

E REPOSITORY PROTOCOLS AND DETAILED RESULTS

All methods receive byte-identical source repositories, common initial translations, task requirements, and acceptance procedures. The time series translation has three generation calls totaling 163,661 tokens. The recommendation translation uses 20 calls totaling 368,368 tokens. Generation proceeds from the public import structure and includes failed or truncated attempts. Each method’s reported total contains one complete copy of this generation cost together with every repair call. Table 6 separates input and output usage in repair.

Table 6: Complete costs for the repository comparison. Total tokens include repair input and output plus common initial translation.

Method	Repair calls	Submissions	Repair input tokens	Repair output tokens	Total tokens
<i>Time series</i>					
LaDiM	35	2	2,345,473	105,096	2,614,230
SWE-agent	80	1	7,984,797	51,367	8,199,825
MatchFixAgent	69	2	13,306,003	128,114	13,597,778
<i>Recommendation</i>					
LaDiM	80	4	8,147,846	295,032	8,811,246
SWE-agent	80	1	7,889,936	58,248	8,316,552
MatchFixAgent	80	1	8,215,327	95,385	8,679,080

The time series protocol compares the core model for three consecutive Adam steps from corresponding source parameters, with dropout disabled and input shape $4 \times 20 \times 1$. Preprocessing and prediction absolute tolerances are 10^{-9} and 10^{-4} . Loss absolute difference, gradient vector L2 difference, and relative parameter update tolerances are 0.02, 0.05, and 0.03. Learning rate behavior is checked around its schedule boundaries. The public evaluation executes the 100-epoch training workflow, notebook cells, and teaching snippets in their source order. Each entry point must produce the required predictions and plots and execute native differentiation. The notebook and snippets provide execution coverage, while the core model supplies paired numerical training measurements. All three methods pass 23 paired checks per seed, giving 69/69 across the public and two confirmation seeds.

The recommendation protocol covers six model paths that execute in the source framework with three sequential Adam updates. Source exceptions and empty returns are checked against the corresponding source behavior. Both the numerical worker and training command invoke the candidate’s optimizer factory with learning rate 0.001. The source Adam settings use decay coefficients 0.9 and 0.999 and epsilon 10^{-8} . Representation absolute tolerance is 10^{-4} . Loss absolute/relative tolerances are $0.002/10^{-4}$, gradient tolerances are $10^{-4}/0.005$, and relative parameter update tolerance is 0.03. The evaluation includes all ten original tests and one epoch of the training command on 16 synthetic examples.

Table 7 reports the declared checks on the public seed. The 114 numerical comparisons comprise 108 checks over six paths, three steps, and six training signals, plus six inference retrieval checks. Twenty-four execution, step count, and native differentiation checks and seven source behavior checks bring the paired total to 145. SWE-agent and MatchFixAgent each lack one retrieval measurement after reward model inference fails. That entry remains n/a in both the 114 and 145 de-

nominators. MatchFixAgent’s original tests are also n/a because collection fails. No method reaches the two confirmation seeds on this repository.

Table 7: Recommendation repository checks passed on the public seed. The numerical checks are a subset of paired checks, and the six training signal rows break down the training checks. One unmeasured retrieval check remains in each baseline’s denominator.

Check	LaDiM	SWE-agent	MatchFixAgent
Paired checks	130/145	104/145	129/145
Numerical checks	99/114	74/114	99/114
Training checks	93/108	69/108	94/108
Inference retrieval	6/6	5/6	5/6
User representations	18/18	16/18	18/18
Item representations	6/18	6/18	6/18
Forward loss	18/18	18/18	18/18
Training return loss	18/18	0/18	18/18
Gradients	18/18	17/18	18/18
Parameter updates	15/18	12/18	16/18
Original tests	10/10	10/10	n/a
Training command	Passed	Passed	Passed
File coverage	Passed	Passed	Failed
Documentation and dependencies	Passed	Passed	Failed

F REPOSITORY CONTEXT AND EVIDENCE

The repository extension separates the version of its dependency plan from the version of the candidate files. Every recorded test stores the candidate version on which it ran. An actual change to candidate code makes earlier measurements stale, and a new result replaces the preceding result for the same test even when the new test fails. Work-unit checkpoints additionally depend on their declared file scopes, test selections, and prerequisite units. A checkpoint without a selected test records syntax validation only. The final acceptance decision uses the complete repository evaluation.

Full tool observations, file reads, and conversations are archived outside the editable workspace. Records carry content hashes and can be retrieved by identifier, JSON pointer, and character range. When the context is rebuilt, it retains the current dependency plan and generation status, the complete structured hypotheses and next steps, the latest measurements, and the last tool batch. Cached code consists only of content returned by earlier reads, with matching current file hashes and relevance to the active unit. The cache contains at most 120,000 characters. Reconstruction is triggered by an independent role handoff, an actual work-unit switch, or messages exceeding 75% of the configured context capacity. Ordinary stage continuation and the first selection of a work unit retain the existing conversation. None of these operations resets accumulated usage.

For the repository experiments, diagnosis allows up to six calls and 64,000 output tokens. The remaining repair calls and output budget are allocated over the remaining external submissions, with at most 16 calls in the first repair stage. The cumulative limits apply to the whole repository.